Page Object Model + BDD Gherkin

avec Playwright & TypeScript

Projet SauceDemo — Formation AmbientIT

Partie 1

Le Pattern POM

Concepts fondamentaux pour débutants

Pourquoi a-t-on besoin de POM ?

Imagine un test classique sans organisation :

```
test("login réussi", async ({ page }) => {
  await page.goto("https://www.saucedemo.com");
  await page.fill("#user-name", "standard_user");
  await page.fill("#password", "secret_sauce");
  await page.click("#login-button");
  await expect(page).toHaveURL(/inventory/);
});

test("login échoué", async ({ page }) => {
  await page.goto("https://www.saucedemo.com");
  await page.fill("#user-name", "wrong");          // copié-collé
  await page.fill("#password", "wrong");            // copié-collé
  await page.click("#login-button");               // copié-collé
  await expect(page.locator(".error")).toBeVisible();
});
```

❌ Si le sélecteur `#user-name` change → on doit modifier **tous** les tests
❌ Logique dupliquée partout
❌ Tests difficiles à lire et à maintenir

Le principe POM en une image

```
┌──────────────────────────────────────────────────────────┐
│                       VOS TESTS                          │
│                                                          │
│  test("login", async ({ loginPage }) => {                │
│    await loginPage.goto()                                │
│    await loginPage.login("user", "pass")  ←── méthodes   │
│    await loginPage.assertOnInventoryPage()    lisibles   │
│  })                                                      │
└──────────────────────────────────────────────────────────┘
                       │  appelle
                       ▼
┌──────────────────────────────────────────────────────────┐
│                   PAGE OBJECT                            │
│                  LoginPage.ts                            │
│                                                          │
│  Sélecteurs :  usernameInput, passwordInput, loginBtn    │
│  Actions    :  goto(), login()                           │
│  Assertions :  assertErrorMessage(), assertOnLoginPage() │
└──────────────────────────────────────────────────────────┘
                       │  interagit avec
                       ▼
┌──────────────────────────────────────────────────────────┐
│                  NAVIGATEUR                              │
│          https://www.saucedemo.com                       │
└──────────────────────────────────────────────────────────┘
```

Anatomie d'un Page Object

```
import { Page, expect } from "@playwright/test";

export class LoginPage {

  // ① SÉLECTEURS — privés, cachés aux tests
  private readonly usernameInput = this.page.getByPlaceholder("Username");
  private readonly passwordInput = this.page.getByPlaceholder("Password");
  private readonly loginButton   = this.page.getByRole("button", { name: "Login" });

  // ② CONSTRUCTEUR — reçoit l'objet page de Playwright
  constructor(private readonly page: Page) {}

  // ③ ACTIONS — ce que l'utilisateur fait
  async goto() {
    await this.page.goto("/");
  }

  async login(username: string, password: string) {
    await this.usernameInput.fill(username);
    await this.passwordInput.fill(password);
    await this.loginButton.click();
  }

  // ④ ASSERTIONS — ce qu'on vérifie
  async assertOnLoginPage() {
    await expect(this.loginButton).toBeVisible();
  }
}
```

Les 3 responsabilités d'un Page Object

① **Sélecteurs** (privés)

```
private readonly usernameInput =
  this.page.getByPlaceholder("Username");
```

→ Un seul endroit à modifier si l'UI change

② **Actions** (verbes)

→ Traduit une intention métier en actions

```
async login(user: string, pass: string) {
  await this.usernameInput.fill(user);
  await this.passwordInput.fill(pass);
  await this.loginButton.click();
}
```

③ **Assertions** (vérifications)

```
async assertErrorMessage(text: string) {
  await expect(this.errorMessage).toContainText(text);
}
```

→ Encapsule les `expect()` pour qu'ils soient réutilisables

💡 **Règle d'or :** un test ne doit jamais voir de sélecteur CSS. Il ne parle qu'en *intentions* : `loginPage.login()`, pas `page.fill("#user-name")`

Bénéfices concrets du POM

Situation	Sans POM	Avec POM
Sélecteur qui change	Modifier N tests	Modifier 1 Page Object
Nouvelle fonctionnalité	Dupliquer du code	Ajouter une méthode
Test qui échoue	Chercher dans le code	Méthode clairement nommée
Onboarding équipe	Difficile à comprendre	Se lit comme du français

```
SANS POM                          AVEC POM
──────────────────────────        ──────────────────────────────
await page.fill("#user-name",     await loginPage.login(
  "standard_user")                  USERS.standard.username,
await page.fill("#password",        USERS.standard.password
  "secret_sauce")                 )
await page.click("#login-button")
```

Partie 2

Architecture du projet SauceDemo

Comment tout s'articule

Structure complète du projet

```
projet-pom-saucedemo/
│
├── src/
│   ├── pages/              ← Les Page Objects
│   │   ├── LoginPage.ts
│   │   ├── InventoryPage.ts
│   │   ├── CartPage.ts
│   │   └── CheckoutPage.ts
│   ├── fixtures/
│   │   └── index.ts        ← Injection automatique des POM
│   └── data/
│       └── users.ts        ← Données de test centralisées
│
├── tests/                  ← Tests Playwright classiques (POM)
│   ├── login.spec.ts
│   ├── inventory.spec.ts
│   └── checkout.spec.ts
│
├── e2e/                    ← Tests BDD Gherkin
│   ├── features/
│   │   ├── login.feature
│   │   ├── inventory.feature
│   │   └── checkout.feature
│   └── steps/
│       ├── login.steps.ts
│       ├── inventory.steps.ts
│       └── checkout.steps.ts
│
└── playwright.config.ts    ← Configuration des deux projets
```

Couche 1 — Les Page Objects

Chaque page de SauceDemo a son Page Object :

```
saucedemo.com/              →  LoginPage.ts
  ↳ /inventory.html         →  InventoryPage.ts
      ↳ /cart.html           →  CartPage.ts
          ↳ /checkout-step-one.html  →  CheckoutPage.ts
```

```typescript
// LoginPage.ts — sélecteurs + actions + assertions
export class LoginPage {
  private readonly usernameInput = this.page.getByPlaceholder("Username");
  // ...
  async login(username: string, password: string) { ... }
  async assertOnLoginPage() { ... }
}
```

```typescript
// InventoryPage.ts — 8 sélecteurs, 4 actions, 4 assertions
export class InventoryPage {
  private readonly productItems = this.page.locator("[data-test='inventory-item']");
  // ...
  async addProductToCart(index = 0) { ... }
  async assertProductCount(count: number) { ... }
}
```

Couche 2 — Les Fixtures

Le mécanisme **d'injection de dépendances** de Playwright :

```
// src/fixtures/index.ts
import { test as base } from "@playwright/test";

type Pages = {
  loginPage:     LoginPage;   // ← types disponibles dans les tests
  inventoryPage: InventoryPage;
  cartPage:      CartPage;
  checkoutPage:  CheckoutPage;
};

export const test = base.extend<Pages>({
  // Playwright instancie automatiquement chaque POM et l'injecte
  loginPage:     async ({ page }, use) => use(new LoginPage(page)),
  inventoryPage: async ({ page }, use) => use(new InventoryPage(page)),
  cartPage:      async ({ page }, use) => use(new CartPage(page)),
  checkoutPage:  async ({ page }, use) => use(new CheckoutPage(page)),
});
```

💡 `base.extend()` = on enrichit le `test` standard de Playwright avec nos propres fixtures. Chaque test peut alors déclarer les POMs dont il a besoin dans sa signature.

Couche 3 — Les Tests POM

Les tests sont lisibles comme une spécification fonctionnelle :

```
// tests/login.spec.ts
import { test } from "../src/fixtures";  // ← notre test enrichi
import { USERS } from "../src/data/users";

test("Scenario: Connexion réussie", async ({
  loginPage,        // ← injecté automatiquement par la fixture
  inventoryPage,    // ← idem
}) => {
  // Given : je suis sur la page de login
  await loginPage.goto();

  // When : je me connecte
  await loginPage.login(USERS.standard.username, USERS.standard.password);

  // Then : je suis redirigé vers les produits
  await inventoryPage.assertOnInventoryPage();
});
```

Pas un seul sélecteur dans le test — **uniquement des intentions métier**.

Couche 4 — BDD Gherkin

La même logique, exprimée en **langage naturel** :

```
# e2e/features/login.feature
Feature: Authentification sur SauceDemo

  Scenario: Connexion réussie avec standard_user
    Given je suis sur la page de login
    When  je me connecte avec les identifiants valides
    Then  je suis redirigé vers la page des produits
```

```
// e2e/steps/login.steps.ts
import { createBdd } from "playwright-bdd";
import { test } from "../../src/fixtures";  // ← même fixture !

const { Given, When, Then } = createBdd(test);

Given("je suis sur la page de login", async ({ loginPage }) => {
  await loginPage.goto();  // ← même Page Object !
});

When("je me connecte avec les identifiants valides", async ({ loginPage }) => {
  await loginPage.login(USERS.standard.username, USERS.standard.password);
});
```

Vue d'ensemble : flux complet

```
.feature (Gherkin)          Step Definitions          Page Objects
__________________          ________________          ____________

Scenario: Login             Given("je suis sur        LoginPage
  Given je suis sur    →      la page de login",  →    .goto()
    la page de login          async({loginPage})
                                => loginPage.goto()
                              )

  When je me connecte    →  When("je me connecte  →   LoginPage
    avec les identifiants      ...", async          →   .login(
    valides                      ({loginPage})           username,
                                 => loginPage            password
                                    .login(...)          )
                              )

  Then je suis           →  Then("je suis         →   InventoryPage
    redirigé vers les         redirigé...",             .assertOn
    produits                  async({inventoryPage})    InventoryPage()
                              => inventoryPage
                                 .assertOn...()
                              )
```

Les fixtures font le lien entre les steps et les Page Objects.

Comment lancer les tests

```
# Tests POM classiques (tests/)
npm test

# Tests BDD Gherkin (e2e/)
npm run test:bdd

# En mode navigateur visible
npm run test:bdd:headed

# Rapport HTML
npm run report
```

```
# Détail de test:bdd :
# 1. bddgen  → lit les .feature, génère e2e/.features-gen/*.spec.ts
# 2. playwright test --project=bdd:chromium  → exécute ces specs
```

💡 Les deux suites de tests (`tests/` et `e2e/`) testent exactement la **même** application avec les **mêmes** Page Objects. Le BDD ajoute une couche de lisibilité pour les parties prenantes non-techniques.

Partie 3

Exercices pas à pas

Reproduire le projet de zéro

Exercice 1 — Créer un Page Object

Objectif : écrire `LoginPage.ts` depuis une page blanche.

Contexte : la page `https://www.saucedemo.com` a :

- Un champ `Username` (placeholder)
- Un champ `Password` (placeholder)
- Un bouton `Login`
- Un div `[data-test="error"]` pour les erreurs

À faire :

1. Créer `src/pages/LoginPage.ts`
2. Déclarer les 4 sélecteurs comme attributs privés
3. Écrire le constructeur `constructor(private readonly page: Page)`
4. Implémenter `goto()` → navigue vers `"/"`
5. Implémenter `login(username, password)` → remplit et clique
6. Implémenter `assertErrorMessage(text)` → vérifie le message d'erreur
7. Implémenter `assertOnLoginPage()` → vérifie l'URL et le bouton

Exercice 1 — Solution

```
import { Page, expect } from "@playwright/test";

export class LoginPage {
  private readonly usernameInput = this.page.getByPlaceholder("Username");
  private readonly passwordInput = this.page.getByPlaceholder("Password");
  private readonly loginButton   = this.page.getByRole("button", { name: "Login" });
  private readonly errorMessage  = this.page.locator("[data-test='error']");

  constructor(private readonly page: Page) {}

  async goto() {
    await this.page.goto("/");
  }

  async login(username: string, password: string) {
    await this.usernameInput.fill(username);
    await this.passwordInput.fill(password);
    await this.loginButton.click();
  }

  async assertErrorMessage(text: string) {
    await expect(this.errorMessage).toBeVisible();
    await expect(this.errorMessage).toContainText(text);
  }

  async assertOnLoginPage() {
    await expect(this.loginButton).toBeVisible();
    await expect(this.page).toHaveURL("/");
  }
}
```

Exercice 2 — Créer la Fixture

Objectif : comprendre et écrire `src/fixtures/index.ts`.

Rappel du mécanisme :

```
// base.extend<T>() crée un nouveau objet test qui expose les fixtures de type T
const test = base.extend<{ loginPage: LoginPage }>({
  loginPage: async ({ page }, use) => {
    //                    ^^^^ page Playwright injecté automatiquement
    //                         ^^^^ appel avec notre instance → injection
    use(new LoginPage(page));
  }
});
```

À faire :

1. Créer `src/fixtures/index.ts`
2. Importer `test as base` depuis `@playwright/test`
3. Déclarer le type `Pages` avec les 4 POMs
4. Créer `export const test = base.extend<Pages>({...})`
5. Définir la fixture pour chaque POM (pattern : `async ({ page }, use) => use(new XxxPage(page))`)
6. Re-exporter `expect` depuis `@playwright/test`

Exercice 2 — Solution

```
import { test as base } from "@playwright/test";
import { LoginPage }     from "../pages/LoginPage";
import { InventoryPage } from "../pages/InventoryPage";
import { CartPage }      from "../pages/CartPage";
import { CheckoutPage }  from "../pages/CheckoutPage";

type Pages = {
  loginPage:     LoginPage;
  inventoryPage: InventoryPage;
  cartPage:      CartPage;
  checkoutPage:  CheckoutPage;
};

export const test = base.extend<Pages>({
  loginPage:     async ({ page }, use) => use(new LoginPage(page)),
  inventoryPage: async ({ page }, use) => use(new InventoryPage(page)),
  cartPage:      async ({ page }, use) => use(new CartPage(page)),
  checkoutPage:  async ({ page }, use) => use(new CheckoutPage(page)),
});

export { expect } from "@playwright/test";
```

💡 Grâce à TypeScript, si tu oublies un fixture ou tu fais une faute de frappe dans un test (`logintPage`), le compilateur te prévient immédiatement.

Exercice 3 — Écrire un Test POM

Objectif : écrire le test de connexion réussie.

Contraintes :

- Importer `test` depuis `../src/fixtures` (pas de `@playwright/test`)
- Utiliser `USERS.standard` depuis `src/data/users.ts`
- Structurer avec les commentaires `// Given / When / Then`
- Ne **jamais** utiliser de sélecteur CSS dans le test

Scénarios à implémenter :

#	Titre	Given	When	Then
1	Connexion réussie	sur la page login	login standard_user	redirigé produits
2	Compte bloqué	sur la page login	login locked_out_user	message d'erreur
3	Mauvais mot de passe	sur la page login	login avec MAUVAIS_MDP	message d'erreur
4	Déconnexion	connecté	logout	redirigé login

Exercice 3 — Solution (extrait)

```
import { test, expect } from "../src/fixtures";
import { USERS }         from "../src/data/users";

test.describe("Feature: Authentification sur SauceDemo", () => {

  test("Scenario: Connexion réussie avec standard_user", async ({
    loginPage,
    inventoryPage,
  }) => {
    // Given : je suis sur la page de login
    await loginPage.goto();

    // When : je me connecte avec des identifiants valides
    await loginPage.login(USERS.standard.username, USERS.standard.password);

    // Then : je suis redirigé vers la page des produits
    await inventoryPage.assertOnInventoryPage();
  });

  test("Scenario: Connexion échouée avec locked_out_user", async ({
    loginPage,
  }) => {
    await loginPage.goto();
    await loginPage.login(USERS.locked.username, USERS.locked.password);
    await loginPage.assertErrorMessage("Sorry, this user has been locked out");
    await loginPage.assertOnLoginPage();
  });

});
```

Exercice 4 — Écrire un fichier `.feature`

Objectif : traduire le test login en Gherkin.

Règles Gherkin à respecter :

- `Feature:` → nom de la fonctionnalité
- `Scenario:` → un cas de test
- `Given` → état initial (contexte)
- `When` → action de l'utilisateur
- `Then` → résultat attendu
- `And` → continuation du précédent mot-clé
- `Background:` → Given commun à tous les scénarios

À faire : traduire ces 4 scénarios en Gherkin dans `e2e/features/login.feature`

> 💡 Chaque phrase Gherkin deviendra une step definition. Sois **cohérent** dans la formulation : "*je suis sur la page de login*" doit être écrit exactement pareil dans chaque scénario qui l'utilise.

Exercice 4 — Solution

```
Feature: Authentification sur SauceDemo

  Scenario: Connexion réussie avec standard_user
    Given je suis sur la page de login
    When  je me connecte avec les identifiants valides
    Then  je suis redirigé vers la page des produits

  Scenario: Connexion échouée avec locked_out_user
    Given je suis sur la page de login
    When  je me connecte avec un compte bloqué
    Then  un message d'erreur "Sorry, this user has been locked out" s'affiche
    And   je reste sur la page de login

  Scenario: Connexion échouée avec un mauvais mot de passe
    Given je suis sur la page de login
    When  je me connecte avec un mauvais mot de passe
    Then  un message d'erreur "Username and password do not match" s'affiche

  Scenario: Déconnexion réussie
    Given je suis connecté en tant que standard_user
    When  je me déconnecte
    Then  je suis redirigé vers la page de login
```

Exercice 5 — Écrire les Step Definitions

Objectif : implémenter les steps du fichier `.feature` précédent.

Le patron à suivre :

```
import { createBdd } from "playwright-bdd";
import { test }       from "../../src/fixtures";  // ← fixture custom

const { Given, When, Then } = createBdd(test);
//                                        ^^^^ injecte nos Page Objects

Given("texte exact du .feature", async ({ loginPage }) => {
  //                                     ^^^^^^^^^^ injecté automatiquement
  await loginPage.goto();
});
```

À faire :

1. Créer `e2e/steps/login.steps.ts`
2. Implémenter chaque step du `.feature`
3. Pour la step avec paramètre `{string}` (message d'erreur), déclarer le paramètre dans la fonction
4. Vérifier que le texte des steps correspond **exactement** aux phrases du `.feature`

Exercice 5 — Solution

```
import { createBdd } from "playwright-bdd";
import { test }       from "../../src/fixtures";
import { USERS }      from "../../src/data/users";

const { Given, When, Then } = createBdd(test);

Given("je suis sur la page de login", async ({ loginPage }) => {
  await loginPage.goto();
});

Given("je suis connecté en tant que standard_user", async ({ loginPage }) => {
  await loginPage.goto();
  await loginPage.login(USERS.standard.username, USERS.standard.password);
});

When("je me connecte avec les identifiants valides", async ({ loginPage }) => {
  await loginPage.login(USERS.standard.username, USERS.standard.password);
});

When("je me connecte avec un compte bloqué", async ({ loginPage }) => {
  await loginPage.login(USERS.locked.username, USERS.locked.password);
});

// Paramètre {string} → capturé automatiquement par Cucumber expressions
Then("un message d'erreur {string} s'affiche", async ({ loginPage }, message: string) => {
  await loginPage.assertErrorMessage(message);
});
```

Exercice 6 (bonus) — Background et Scenario Outline

Background — Given commun à tous les scénarios d'une Feature :

```
Feature: Catalogue produits

  Background:
    Given je suis connecté en tant que standard_user
    # Ce Given s'exécute avant CHAQUE Scenario de cette Feature

  Scenario: La liste affiche 6 produits
    Then la liste contient 6 produits   # pas besoin de re-dire "connecté"
```

Scenario Outline — paramétrer un scénario :

```
  Scenario Outline: Tri des produits
    When  je trie les produits par <critere>
    Then  les produits sont triés correctement

    Examples:
      | critere          |
      | prix croissant   |
      | prix décroissant |
      | nom A vers Z     |
      | nom Z vers A     |
```

💡 `Scenario Outline` + `Examples` = un seul scénario exécuté N fois avec des données différentes. Équivalent de `test.each()` en Playwright.

Récapitulatif — Ce que vous avez appris

```
┌──────────────────────────────────────────────────────────┐
│                    PYRAMIDE DU PROJET                    │
│                                                          │
│  ┌──────────────────────────────────┐                    │
│  │  .feature (Gherkin)              │  ← Langage métier  │
│  │  login.feature                   │    lisible par tous│
│  ├──────────────────────────────────┤                    │
│  │  Step Definitions                │  ← Colle entre     │
│  │  login.steps.ts                  │    Gherkin et POM  │
│  ├──────────────────────────────────┤                    │
│  │  Fixtures                        │  ← Injection       │
│  │  src/fixtures/index.ts           │    automatique     │
│  ├──────────────────────────────────┤                    │
│  │  Page Objects                    │  ← Sélecteurs      │
│  │  LoginPage / InventoryPage ...   │    et actions      │
│  ├──────────────────────────────────┤                    │
│  │  Data                            │  ← Données de test │
│  │  src/data/users.ts               │    centralisées    │
│  └──────────────────────────────────┘                    │
└──────────────────────────────────────────────────────────┘
```

POM = un seul endroit pour les sélecteurs

Fixtures = injection automatique, pas de `new` dans les tests

Gherkin = spécification exécutable, lisible par le métier

Merci !

Ressources

- **Playwright docs** → playwright.dev
- **playwright-bdd** → vitalets.github.io/playwright-bdd
- **Gherkin syntax** → cucumber.io/docs/gherkin

```
# Pour lancer les tests du projet :
npm test            # POM classique
 npm run test:bdd    # BDD Gherkin
 npm run report      # Rapport HTML
```